esprit
Se former autrement



SOLID PATTERN

Cours présenté par Tarek Ayari

PRINCIPES SOLID

Définition

-Solid est une abréviation pour les cinq premiers principes de la conception orientée objet.

-Ces principes, lorsqu'ils sont combinés ensemble, il est facile pour les développeurs de créer des logiciels faciles à entretenir et à étendre.

-Ils sont aussi une partie du développement logiciel agile.

PRINCIPES SOLID

5 principes

- **S**RP : **S**ingle Responsibility Principle
  - Une classe doit avoir une et une seule responsabilité
- **O**CP : **O**pen/Close Principle
  - Une entité doit être ouverte aux extensions et fermée aux modifications
- **L**SP : **L**iskov Substitution Principle
  - Les sous-types doivent être interchangeables par leurs types de base
- **I**SP : **I**nterface Segregation Principle
  - Un client ne doit pas être forcé de dépendre de méthodes qu'il n'utilise pas
- **D**IP : **D**ependency Inversion Principle
  - Il faut dépendre des abstractions, pas des implémentations

SRP : SINGLE RESPONSIBILITY PRINCIPLE

Le principe de responsabilité unique, est qu'une classe donnée ne doit avoir qu'une seule responsabilité, et, par conséquent, qu'elle ne doit avoir qu'une seule raison de changer.

Les avantages de cette approche sont les suivants :

- Diminution de la complexité du code
- Augmentation de la lisibilité de la classe
- Meilleure encapsulation, et meilleure cohésion, les responsabilités étant regroupées.

SRP : SINGLE RESPONSIBILITY PRINCIPLE



SRP : EXEMPLE CONCRET

ENONCE



Document
Microsoft Word

CORRECTION



Document
Microsoft Word

OCP : OPEN/CLOSE PRINCIPLE

Une classe doit être ouverte à l'extension, mais fermée à la modification

- En effet, vous ne devez pas modifier le code courant quand une nouvelle fonctionnalité doit être ajoutée- vous devriez plutôt être en mesure d'étendre les types et mettre en œuvre la fonctionnalité.
- La conformité au principe Ouvert Fermé facilite la création d'applications qui sont réutilisables et peuvent être maintenues facilement.
- *Vous devriez être en mesure de représenter ce principe en utilisant des* ***abstractions.***
- *Les dérivés qui sont créés à partir de cette abstraction sont clos pour modification depuis l'abstraction est fixé. Cependant, vous pouvez étendre le comportement en créant de nouveaux dérivés de l'abstraction qui a déjà été défini.*

OCP : OPEN/CLOSE PRINCIPLE

ENONCE



Document
Microsoft Word

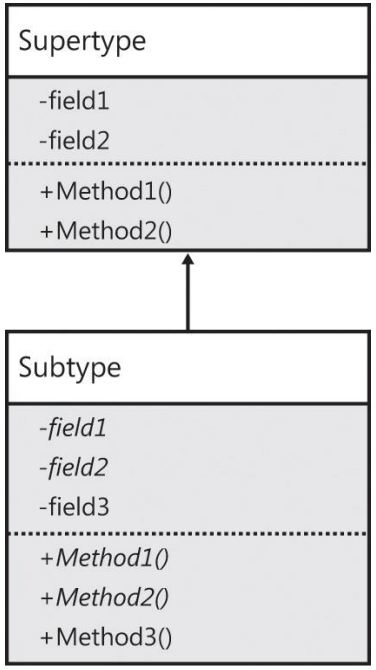
CORRECTION



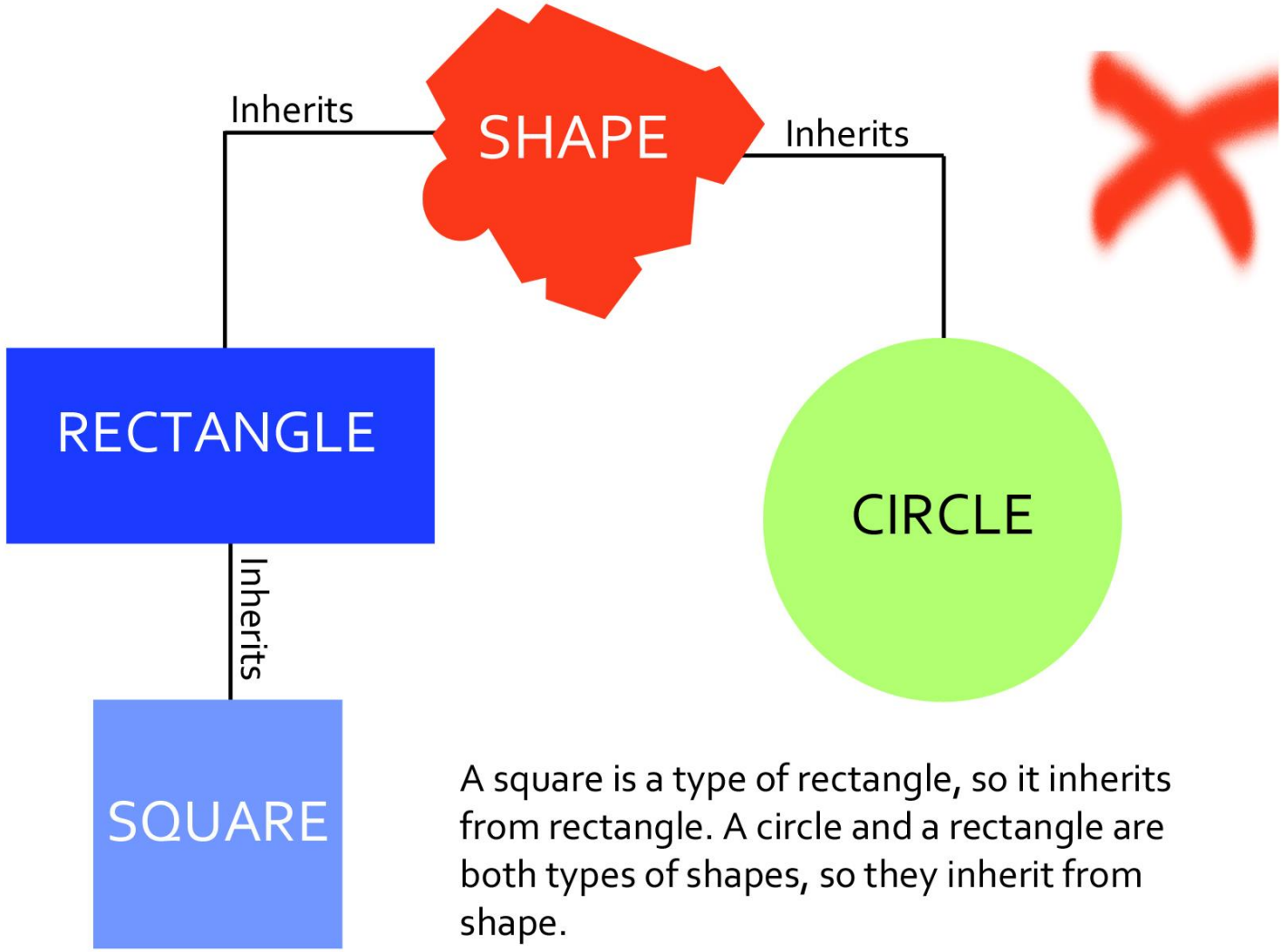
Document
Microsoft Word

LSP : LISKOV SUBSTITUTION PRINCIPLE

- il doit être possible de substituer une classe "parente" par l'une de ses classes enfants sans créer un probleme
- *Pour le mettre en pratique et le vérifier, il suffit de se demander si une sous classe peut facilement remplacer la classe de base sans créer un problème.*



LSP : LISKOV SUBSTITUTION PRINCIPLE



A square is a type of rectangle, so it inherits from rectangle. A circle and a rectangle are both types of shapes, so they inherit from shape.

LSP : LISKOV SUBSTITUTION PRINCIPLE

Soit une classe Rectangle représentant les propriétés d'un rectangle : hauteur, largeur.

```php
class Rectangle
{
    private $largeur;
    private $hauteur;

    public function setHauteur($hauteur)
    {
        $this->hauteur = $hauteur;
    }

    public function getHauteur()
    {
        return $this->hauteur;
    }

    public function setLargeur($largeur)
    {
        $this->largeur = $largeur;
    }

    public function getLargeur()
    {
        return $this->largeur;
    }
}
```

LSP : LISKOV SUBSTITUTION PRINCIPLE

- Soit une classe Carré que l'on fait dériver de la classe Rectangle.
- En effet, en mathématiques, un carré est un rectangle.
- Donc, on définit naturellement la classe Carré comme sous-type de la classe Rectangle

```php
class Carre extends Rectangle
{
   //$this->hauteur représentera mon "coté" et $this->largeur ne sera donc plus jamai

    public function setLargeur($largeurEtHauteur)
    {
        //Tant pis pour largeur qui sera toujours vide :(
        $this->setHauteur($largeurEtHauteur);
    }

    public function getLargeur()
    {
        return $this->getHauteur();
    }
}
```

LSP : LISKOV SUBSTITUTION PRINCIPLE

- *La question est alors, le carré peut-il remplacer le rectangle dans le calcul de surface ? Sans créer un probleme ?*

- *Eh bien non !*

- *Si vous remplacez l'objet de type rectangle et que vous mettez un objet de type carré.*
- *Avec quel coté du rectangle peut-on calculer la surface de cet objet vu que le carré ne possède qu'un seul coté, qu'une seule propriété, et que le type rectangle en possède deux ?*

- *Ceci est une violation du principe de substitution de Liskov.*

ISP : INTERFACE SEGREGATION PRINCIPLE

- Un client ne doit pas être forcé de dépendre de méthodes qu'il n'utilise pas
- Préférer plusieurs interfaces spécifiques pour chaque client plutôt qu'une seule interface générale
- Créer des interfaces à grains fin qui sont spécifiques au client
- *Le principe ISP indique que les clients ne devraient pas être contraints de mettre en œuvre une interface qui contient des déclarations de membres ou opérations qu'ils ne seraient pas besoin ou ne jamais utiliser. Ces interfaces sont connues comme «gras» ou interfaces pollués*

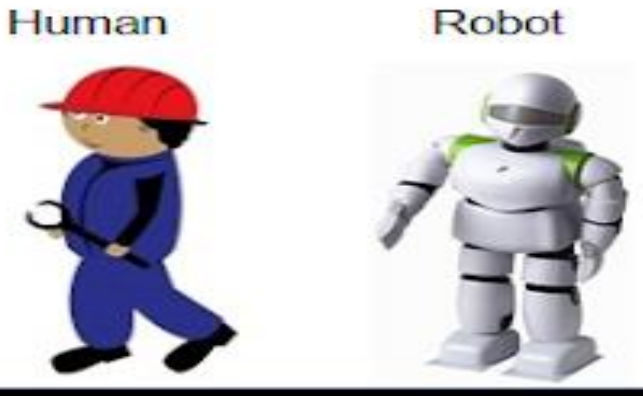
ISP : INTERFACE SEGREGATION PRINCIPLE

Interface Segregation Principle

"Clients should not be forced to depend upon interfaces that they don't use"



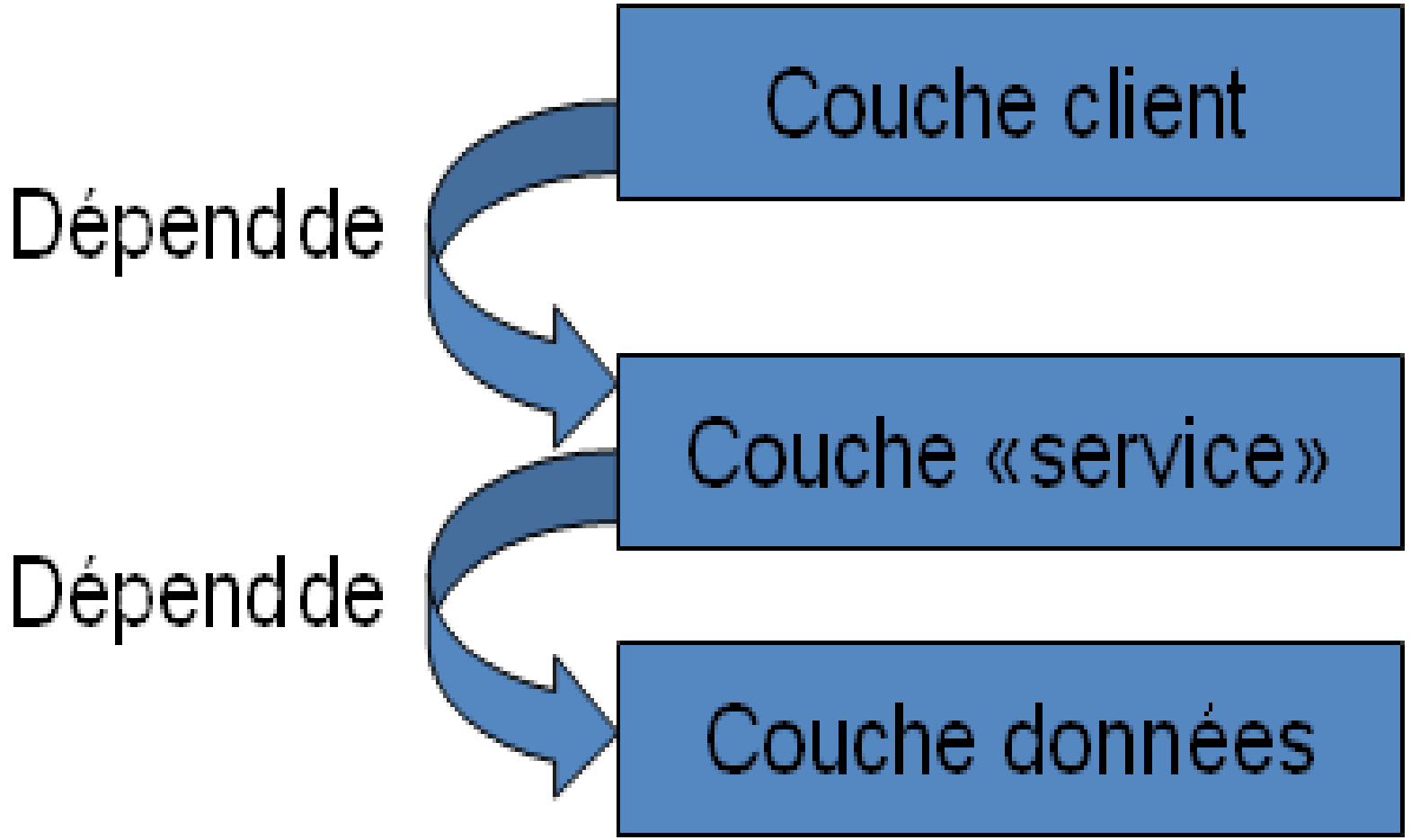
Iworker has methods that are different for different workers and violates ISP



DIP : DEPENDENCY INVERSION PRINCIPLE

- Il faut dépendre des abstractions, pas des implémentations
- Une caractéristique clé du DIP est la programmation d'abstraction pour que les consommateurs puissent compter sur ces abstractions plutôt que leurs implémentations.
- L'idée est que chaque point de contact entre deux modules soit matérialisé par une abstraction
- Une abstraction se matérialise par une interface ou une classe de base qui est aussi l'abstraction de classe plus élevé
- le DIP réduit couplage entre différents morceaux de code.

DIP : EXEMPLE CONCRET



CONCLUSION

• Il est possible de ne pas appliquer tous les principes SOLID d'un coup, il n'est pas choquant d'ajuster les choix de conception au moment où l'on en a besoin .

• La raison et l'expérience vont nous permettre d'arbitrer. Avec l'expérience vous pourrez appliquer systématiquement SOLID avec le même effort, mais probablement pas dès le début